

Technical Whitepaper

The RoundFi protocol end to end — pool lifecycle, the Triple Shield, and the economics that make it self-funding

RoundFi
2026-06-23

CONTENTS

1. What the protocol is
2. Pool lifecycle
3. The four vaults
4. Contributions, the split, and the Triple Shield
5. Default settlement
6. Economics: why the pool is self-funding
7. The Escape Valve (secondary market)
8. The pause circuit breaker

Derived document. This is a derivation of [docs/spec/MASTER-SPEC.md](#) (§3 Solution, §4 Protocol, §8 Economics). The Master Spec is the single source of truth; if a number here disagrees with it, the Master Spec wins. Every constant is pinned to the deployed Jun 2026 source (`roundfi-core` , `crates/math/`). Citations point at the source file so this document can be re-verified, not trusted.

1. What the protocol is

RoundFi is a rotating savings-and-credit association (ROSCA — *consórcio / junta* in Brazil) implemented on Solana. The ROSCA is the **data engine**: a fixed set of members each pay a recurring installment, and each cycle exactly one member receives the pooled credit (the **carta**). Every contribution, payout, and default is a Solana transaction, and each consequential event emits an immutable on-chain attestation. The reputation built from that stream is the **product**; this document specifies the **mechanism** that produces it.

The architecture is three layers, and this whitepaper covers the first two — the pool and the attestation substrate — with the third (the reputation engine) specified in the companion [04-behavioral-reputation-score](#).

Layer	What it does
The pool	The on-chain ROSCA. Stake, escrow, and a solidarity reserve protect it against defaults.
The attestation layer	Each event emits a signed, immutable on-chain attestation keyed by subject = wallet .
The reputation engine	Off-chain scorer + on-chain level ladder; higher tier $\Rightarrow$ lower required stake.

All protocol logic lives in the `roundfi-core` program. The mechanics below are its public surface.

2. Pool lifecycle

A pool moves through four states: **Forming** $\rightarrow$ **Active** $\rightarrow$ **Completed** $\rightarrow$ **Closed**. The state machine is deliberately small, and every transition is a named instruction.

Transition	Instruction	Notes
(create) $\rightarrow$ Forming	<code>create_pool</code> + <code>init_pool_vaults</code>	Two ixs: PDA allocation, then the four USDC vault ATAs.
Forming $\rightarrow$ Active	<code>join_pool</code> (last member)	Flips Active when <code>members_joined == members_target</code> .
(per cycle)	<code>contribute</code> , <code>claim_payout</code>	A claim advances <code>current_cycle</code> and re-arms <code>next_cycle_at</code> .
Active $\rightarrow$ Completed	<code>claim_payout</code> (final cycle)	When <code>next_cycle $\geq$ cycles_total</code> .
Completed $\rightarrow$ Closed	<code>close_pool</code>	Pure terminal transition; decrements committed TVL. Moves no funds.

The pool-creation step is **split into two instructions** — `create_pool` allocates the Pool PDA, and `init_pool_vaults` creates the four USDC vault ATAs — because doing both in one instruction would exceed Solana's stack-depth budget.

2.1 The geometry invariant

The protocol enforces a **hard geometry invariant**: `cycles_total == members_target` (`create_pool.rs:123`, SEV-038). Every member slot is drawn **exactly once** over the life of the pool; there are no orphan cycles and no member who pays without ever receiving. This is the structural property that makes the contribution/payout accounting close, and it is checked at allocation time so a malformed pool can never enter `Forming`.

2.2 Cycle advancement is claim-driven, not time-driven

Progress through the pool is **claim-driven**. `claim_payout` of cycle N sets `current_cycle = N+1` and advances `next_cycle_at += cycle_duration`. Time does **not** decide whether the pool can progress — it only governs two things:

1. the **on-time vs late** classification of a contribution, and
2. the **default grace window** (§5).

This separation matters: a slow cycle cannot stall the pool's state machine, and conversely the passage of time alone never forces a transition. The clock is an input to scoring and to default eligibility, not a driver of the lifecycle.

2.3 Contemplation order — who receives, and when

§2.1 fixes **how many** members are contemplated (exactly one per cycle, each exactly once). This section fixes **which** one.

`pool.ordering_policy` selects the mechanism:

Value	Policy	Mechanism
0	arrival	Payout follows join order. A pool created before the field existed reads a zeroed byte and behaves this way.
1	sorteio	<code>finalize_draw</code> draws the order on-chain when the pool fills and freezes it into a <code>DrawResult PDA</code> .

An unrecognized value is **rejected**, not silently treated as arrival — the validation is fail-closed.

The `DrawResult` stores `order[seat] = cycle`, and the fact that this mapping is a **permutation** is what makes §2.1's guarantee structural. "Every slot is drawn exactly once" is not an invariant anyone checks each cycle; it is a property of a bijection between seats and cycles.

2.4 The lance — bidding to be contemplated earlier

A Brazilian *consórcio* contemplates through **two** channels each assembly: the *sorteio* and the **lance** — a bid. RoundFi implements both. The lance changes *who* is contemplated *when*; it never changes *whether* the winner still owes every remaining installment. That distinction is the pay-after-receiving thesis, and it is non-negotiable.

Three mechanisms, each building on the previous:

Prepayment. `contribute` accepts a member's **next unpaid** installment even when it sits ahead of `pool.current_cycle`. There is no skipping (the payment must still be exactly `contributions_paid`) and no back-pay (a member who has fallen behind still fails the gate). Prepaying can only bring float into the pool *earlier*, never reduce it — so it is strictly safer for the viability guard of §6.1, never worse.

Lance embutido (embedded bid). `place_embedded_bid` offers the installments a member has already prepaid as their bid. Depth is

```
depth = contributions_paid - current_cycle - 1
```

and the `-1` is load-bearing: `contributions_paid == current_cycle + 1` means the member is merely **current**, not ahead, so it must not count as bid material. A bid must be **strictly** greater than the standing `pool.current_bid_depth`; ties lose, which makes the outcome independent of transaction ordering within a block.

Lance livre (sealed free bid). `place_bid_commit` writes `sha256(amount || salt || bidder)` into a Bid PDA; `place_bid_reveal` opens it. The bid amount must be an exact multiple of the installment, because a free bid is K installments prepaid in one shot — which is why the whole feature adds no new contemplation math, and why K installments settle with a **single** attestation.

Three properties are worth stating plainly, because each one removed work rather than adding it:

- **Contemplation is a swap.** A winning bid transposes two entries of `DrawResult.order`: the bidder takes the current cycle, and the displaced seat inherits the bidder's former future cycle. A transposition of a permutation is still a permutation — so §2.1 survives untouched, and `claim_payout` required **no changes at all** to support bidding.
- **Anti-sniping is temporal.** Commits require `clock < pool.next_cycle_at`; reveals require `next_cycle_at ≤ clock < next_cycle_at + GRACE_PERIOD_SECS`. The windows are **dis-joint**, so no one can observe a revealed bid and answer it. No cool-down, no extra state — the schedule itself is the defense.
- **Losing costs only the transaction fee.** `place_bid_reveal` checks eligibility **before** moving any tokens, so a losing bid reverts and the USDC never leaves the bidder's wallet. There is consequently no bid vault to hold funds, no refund path to get them back, and no settlement step to run.

The honest trade-off: contemplation-by-bid means a member with more capital can move up the queue. That is inherent to the consórcio model rather than a flaw in this implementation, and it is bounded — the bid is real money paid into the pool, which accelerates contemplation for *everyone*. We document it rather than design it away.

3. The four vaults

Each pool owns **four USDC token accounts**, all with PDA authorities. The separation of funds across four vaults is what lets the protocol reason about solvency precisely — each vault has one job.

Vault	Role
<code>pool_usdc_vault</code>	The pool float : contributions land here net of splits; payouts are drawn from here.
<code>escrow_vault</code>	Member stakes + the escrow slice of each contribution. The vesting / seizure surface.
<code>solidarity_vault</code>	The "Cofre Solidário" — the 1%-of-installment reserve that covers the first tranche of a default.
<code>yield_vault</code>	Idle USDC parked into the yield adapter .

These four vaults are the physical substrate for everything that follows: the contribution split (§4) fans contributions into three of them, the Triple Shield (§4.2) defends out of three of them, and the Yield Cascade (§6.2) moves float through the fourth.

4. Contributions, the split, and the Triple Shield

4.1 The contribution split

Every `installment` is split on `contribute` into three slices, each routed to a different vault:

Slice	bps	Destination
Solidarity	100 (1%)	<code>solidarity_vault</code>
Escrow	<code>escrow_release_bps</code> (default 2500 = 25%)	<code>escrow_vault</code>
Pool float	remainder (default 74%)	<code>pool_usdc_vault</code>

So of every installment, **1% builds the solidarity reserve, 25% accrues to the member's escrow, and the remaining 74% becomes spendable pool float**. The 74% float retention is not an arbitrary number — it is the quantity the viability guard (§6.1) protects, and it is what makes the pool self-funding.

4.2 Stake and the Triple Shield

A member stakes a fraction of the credit at join, sized by their reputation tier (the four-tier ladder — 50% / 25% / 10% / 3% — is specified in the reputation document). That **stake**, plus the **escrow** slice accumulating from each contribution, plus the **solidarity** reserve, form the **Triple Shield**: three layered pools of capital that together make a single member's default **recoverable** for the rest of the pool.

The shield is directional. It defends the pool against a member who stops paying, and it rewards a member who keeps paying:

- **Seizure** (the member defaults) draws *down* the shield to make the pool whole — see §5.
- **Vesting** (the member pays on time) releases the stake *back* to the member. An on-time member progressively unlocks their stake from escrow via `release_escrow(checkpoint)`, gated on `member.on_time_count ≥ checkpoint`. Pay on time and you reclaim your stake in tranches; pay late and the stake stays locked (`EscrowLocked`).

This is the on-chain expression of the protocol's core stance: **discipline is rewarded, delinquency is collateralized**.

5. Default settlement

When a member falls behind and the grace window elapses, any party can call `settle_default(cycle)` to make the pool whole at the defaulter's expense.

5.1 Eligibility

`settle_default(cycle)` requires **all** of:

- `args.cycle == pool.current_cycle` — settling the current cycle only.
- `member.contributions_paid < pool.current_cycle` — the member is **genuinely behind**.
- `now ≥ pool.next_cycle_at + GRACE_PERIOD_SECS` — the grace window has fully elapsed. The window is **7 days on the mainnet build** and **1 day on the devnet-canary build** (`constants.rs`, `cfg-gated`). The window is generous on purpose: **default is a last resort, not a tripwire**.
- `!member.defaulted` — one-shot; a member is settled at most once.
- Pool status is `Active`.

5.2 The seizure order

Settlement draws on the Triple Shield in a **fixed order**, shallowest reserve first, so that the broadly-shared solidarity pool absorbs the first hit and the defaulter's own capital absorbs the rest:

1. **Solidarity vault** — up to the missed installment.
2. **Member escrow balance** — up to the remaining shortfall.
3. **Member stake** — the remainder.

All seized USDC flows into `pool_usdc_vault`, so the pool can **still pay its drawn member** for the cycle. The defaulter is marked `defaulted = true` and a `SCHEMA_DEFAULT` attestation fires, which is what feeds the -500 score penalty in the reputation engine. The solidarity-then-escrow-then-stake ordering is the economic heart of the shield: the pool is protected first by a small mutualized reserve, and only then by the individual's collateral.

6. Economics: why the pool is self-funding

6.1 The Seed Draw viability guard

The split in §4.1 retains 74% of every installment as float. The **Seed Draw guard** (`crates/math/src/seed_draw.rs`) is the inequality that guarantees this float is always enough to pay the drawn member, even at **cycle 0** when no float has yet accumulated:

```
members × installment × (MAX_BPS - solidarity_bps - escrow_release_bps) / MAX_BPS ≥ credit
```

With the defaults that is `members × installment × 0.74 ≥ credit`. `create_pool` **refuses to allocate** any pool whose math doesn't close — it reverts with `PoolNotViable`. Because allocation is blocked unless the inequality holds, the **cycle-0 claim can never under-run the float** (the error it would otherwise hit, `WaterfallUnderflow`, is made unreachable by construction). This is what makes the pool **self-funding**: the float retained from contributions is enough to pay the drawn member every cycle **without any external subsidy**.

6.2 The Yield Cascade

Idle pool float doesn't sit dead. It can be deposited into a yield adapter via `deposit_idle_to_yield` and the realized surplus harvested via `harvest_yield`. The realized yield runs a **PDF-canonical waterfall** — four tranches, in order:

#	Tranche	Sizing	Destination
1	Protocol fee	<code>DEFAULT_FEE_BPS_YIELD</code> = 20% of gross	Treasury — the only physical outflow .
2	Guarantee Fund	logical earmark	<code>pool.guarantee_fund_balance</code> .
3	LP slice	<code>config.lp_share_bps</code> = 65% (default)	<code>pool.lp_distribution_balance</code> ("Anjos de Liquidez").
4	Residual	whatever remains	Stays in <code>pool_usdc_vault</code> as the " prêmio de paciência " (patience premium).

Only tranche 1 actually leaves the pool; tranches 2 and 3 are **logical earmarks** on Pool fields, and tranche 4 simply remains as spendable float. A **slippage floor** — `harvest_yield(min_realized)` — protects against an adapter under-reporting yield. Adapters are **pluggable**: the mock is for devnet, and Kamino is the mainnet target (the on-chain CPI is shipped; the operational reserve pin is pending).

6.3 Stake economics

The stake-discount ladder (**50%** → **25%** → **10%** → **3%** of credit) is the user-facing reward for a proven track record. A member who reaches **Elite (L4)** frees roughly **94%** of the capital an **L1** must lock — a direct, legible incentive to build and keep reputation. Crucially, **the protocol's risk does not rise as the stake falls**, because the upper tiers are gated on a long completed- pool history that is itself the risk signal. The discount is large precisely because earning the right to it is hard; the anti-farming defenses that protect the ladder are specified in the reputation document.

7. The Escape Valve (secondary market)

A member can exit a pool **mid-life** by selling their **position** — the slot, its NFT, and its pending obligations — on a secondary market. There are two listing paths.

7.1 Direct listing

`escape_valve_list(price)` → `escape_valve_buy`. The listing is **immediately Active** and any buyer can take it at the posted price.

7.2 Commit-reveal (anti-MEV)

The direct path posts a price in the clear, which a searcher could snipe. The **commit-reveal** path (issue #232) defends against that:

1. `escape_valve_list_commit(hash)` publishes only `SHA-256(price || salt)`, **hiding the price**.
2. `escape_valve_list_reveal(price, salt)` later publishes the price and arms `buyable_after = now + REVEAL_COOLDOWN_SECS (30s)`.
3. The intended buyer — who already knows `(price, salt)` off-chain — lands their `escape_valve_buy` **at the boundary**, ahead of any searcher reacting to the now-public price. `escape_valve_buy` enforces `now ≥ buyable_after`, reverting with `ListingNotBuyableYet` otherwise.

7.3 What transfers on a buy

On `escape_valve_buy` the handler atomically:

- transfers `price` **buyer** → **seller**;
- **closes the seller's Member PDA** and **creates the buyer's**, carrying over the **position-state snapshot**: `contributions_paid`, `escrow_balance`, `on_time_count`, `late_count`, `slot_index`, and the stake tier;
- thaws → transfers → re-freezes the **position NFT** (mpl-core CPIs via the slot's `position_authority`); and
- closes the listing.

Position-state is not wallet-reputation. The buyer inherits the **slot's** operational state — the pending obligations they must now honor — but **not** the seller's `ReputationProfile` PDA (`[b"reputation", seller_wallet]`), which holds the portable score and tier and **stays with the seller's wallet**. The buyer brings their own profile. Buying a half-paid position assumes that position's remaining obligations; it transfers **no credit history in either direction**, because the reputation engine scores attestations keyed by **subject = wallet**, so reputation naturally tracks the wallet, not the slot.

8. The pause circuit breaker

`pause(bool)` (authority-gated) sets `config.paused`. While paused, **13 instructions** revert with `ProtocolPaused` at **account-validation time** — via the `!config.paused` constraint on the shared `config` account — including `create_pool`, `join_pool`, `contribute`, `claim_payout`, `release_escrow`, `deposit_idle_to_yield`, `harvest_yield`, and **both Escape Valve paths**.

`settle_default` is a **deliberate carve-out**: a default already in flight must remain settleable even while the rest of the protocol is frozen, so that a pause can never trap a pool with an unrecoverable delinquent member. This single exemption is the reason the pause is safe to use as an emergency brake — it stops new activity without stranding obligations that are already due.

Cross-references: program topology, the account/PDA model, and the full instruction surface → [03-architecture-spec](#) ; the reputation engine, the four-tier ladder, and the Pass-3 attestation taxonomy → [04-behavioral-reputation-score](#) ; solvency math under stress → [05-stress-lab-economic-model](#) . The canonical mechanics live in MASTER-SPEC §4 and §8.